

COMICCRAFT AI

Complete Source Code Files

This PDF contains the source code of the generated ComicCraft AI project, organized file-by-file. The original folder structure is preserved in the headings.

run.py

```
import uvicorn

if __name__ == "__main__":
    uvicorn.run("app.main:app", host="127.0.0.1", port=8000, reload=True)
```

requirements.txt

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.34,<1.0
jinja2>=3.1,<4.0
python-multipart>=0.0.20,<1.0
pydantic>=2.10,<3.0
pydantic-settings>=2.7,<3.0
python-dotenv>=1.0,<2.0
google-generativeai>=1.30,<2.0
huggingface_hub>=0.35,<1.0
Pillow>=11.0,<13.0
fpdf2>=2.8,<3.0
httpx>=0.28,<1.0
pytest>=8.0,<9.0
```

.env.example

```
GEMINI_API_KEY=  
GEMINI_OUTLINE_MODEL=gemini-3.8-flash  
GEMINI_STORY_MODEL=gemini-3.8-flash  
HF_TOKEN=  
HF_IMAGE_MODEL=black-forest-labs/FLUX.1-schnell  
DEMO_MODE=true  
PANEL_COUNT=5  
MAX_PROMPT_LENGTH=1200  
APP_NAME=ComicCraft AI
```

app/__init__.py

```
"""ComicCraft AI application package."""
```

app/main.py

```
from pathlib import Path
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from app.config import settings
from app.routes import router

BASE = Path(__file__).resolve().parent
app = FastAPI(title=settings.app_name, version="1.0.0")
app.mount("/static", StaticFiles(directory=str(BASE / "static")), name="static")
templates = Jinja2Templates(directory=str(BASE / "templates"))
app.state.templates = templates
app.include_router(router)
```

app/config.py

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "ComicCraft AI"
    gemini_api_key: str = ""
    gemini_outline_model: str = "gemini-3.8-flash"
    gemini_story_model: str = "gemini-3.8-flash"
    hf_token: str = ""
    hf_image_model: str = "black-forest-labs/FLUX.1-schnell"
    demo_mode: bool = True
    panel_count: int = 5
    max_prompt_length: int = 1200
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

settings = Settings()
```

app/schemas.py

```
from pydantic import BaseModel, Field

class PromptRequest(BaseModel):
    story_prompt: str = Field(min_length=3, max_length=1200)
    character_name: str = Field(default="Mira", min_length=1, max_length=60)
    setting: str = Field(default="a colorful modern city", max_length=200)
    tone: str = Field(default="fun and adventurous", max_length=100)
    art_style: str = Field(default="vibrant cinematic comic art", max_length=120)

class PanelOutline(BaseModel):
    panel_number: int
    title: str
    scene_description: str
    image_prompt: str

class ComicOutline(BaseModel):
    title: str
    panels: list[PanelOutline]

class PanelStory(BaseModel):
    panel_number: int
    caption: str
    narration: str
    dialogue: str

class ComicStory(BaseModel):
    title: str
    panels: list[PanelStory]

class ComicPanel(BaseModel):
    panel_number: int
    title: str
    scene_description: str
    image_prompt: str
    caption: str
    narration: str
    dialogue: str
    image_url: str

class ComicResponse(BaseModel):
    title: str
    panels: list[ComicPanel]
    pdf_url: str | None = None
```

app/routes.py

```
from fastapi import APIRouter, Request
from fastapi.responses import HTMLResponse, JSONResponse, RedirectResponse
from app.schemas import PromptRequest
from app.services.comic_service import ComicService
from app.services.exporters import export_pdf

router = APIRouter()
service = ComicService()
last_result = None

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return request.app.state.templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate", response_class=HTMLResponse)
def generate(request: Request, form: PromptRequest):
    global last_result
    try:
        title, panels = service.generate(form)
        last_result = {"title": title, "panels": panels}
        return request.app.state.templates.TemplateResponse("comic_preview.html", {"request": request, "title":
title, "panels": panels})
    except Exception as exc:
        return request.app.state.templates.TemplateResponse("error.html", {"request": request, "error": str(exc)},
status_code=500)

@router.post("/generate-comic/json")
def generate_json(form: PromptRequest):
    global last_result
    title, panels = service.generate(form)
    last_result = {"title": title, "panels": panels}
    return JSONResponse({"title": title, "panels": [p.model_dump() for p in panels]})

@router.post("/export")
def export():
    if not last_result:
        return RedirectResponse("/", status_code=303)
    url = export_pdf(last_result["title"], last_result["panels"])
    return RedirectResponse(f"/export-success?url={url}", status_code=303)

@router.get("/export-success", response_class=HTMLResponse)
def export_success(request: Request, url: str):
    return request.app.state.templates.TemplateResponse("export_success.html", {"request": request, "url": url})

@router.get("/health")
def health():
    return {"status": "ok"}
```

app/ai/__init__.py

```
"""ComicCraft AI helpers."""
```

app/ai/gemini_flash.py

```
"""Simple Gemini Flash helper used by ComicCraft AI."""
import json
from app.config import settings

class GeminiFlash:
    def __init__(self):
        self._client = None

    def _get_client(self):
        if self._client is None:
            if not settings.gemini_api_key:
                raise ValueError("GEMINI_API_KEY is missing. Add it to .env.")
            from google import genai
            self._client = genai.Client(api_key=settings.gemini_api_key)
        return self._client

    def generate(self, prompt: str) -> str:
        if not prompt.strip():
            raise ValueError("Prompt cannot be empty.")
        response = self._get_client().models.generate_content(
            model=settings.gemini_outline_model,
            contents=prompt,
        )
        if not response.text:
            raise RuntimeError("Gemini returned an empty response.")
        return response.text.strip()

    def generate_json(self, prompt: str) -> dict:
        from google.genai import types
        if not prompt.strip():
            raise ValueError("Prompt cannot be empty.")
        response = self._get_client().models.generate_content(
            model=settings.gemini_outline_model,
            contents=prompt,
            config=types.GenerateContentConfig(response_mime_type="application/json"),
        )
        if not response.text:
            raise RuntimeError("Gemini returned an empty JSON response.")
        return json.loads(response.text)

_gemini = None

def get_gemini() -> GeminiFlash:
    global _gemini
    if _gemini is None:
        _gemini = GeminiFlash()
    return _gemini

def generate_text(prompt: str) -> str:
    return get_gemini().generate(prompt)

def generate_json(prompt: str) -> dict:
    return get_gemini().generate_json(prompt)
```

app/services/gemini_service.py

```
import json
from app.config import settings
from app.schemas import ComicOutline, ComicStory

class GeminiService:
    def __init__(self):
        self._client = None

    def _get_client(self):
        if self._client is None:
            if not settings.gemini_api_key:
                raise RuntimeError("GEMINI_API_KEY is not configured.")
            from google import genai
            self._client = genai.Client(api_key=settings.gemini_api_key)
        return self._client

    def _generate_json(self, prompt: str, model: str, schema):
        from google.genai import types
        response = self._get_client().models.generate_content(
            model=model,
            contents=prompt,
            config=types.GenerateContentConfig(
                response_mime_type="application/json",
                response_schema=schema,
            ),
        )
        if not response.text:
            raise RuntimeError("Gemini returned an empty response.")
        return json.loads(response.text)

    def create_outline(self, request) -> ComicOutline:
        prompt = f"""Create an original five-panel comic outline.
        Story idea: {request.story_prompt}
        Main character: {request.character_name}
        Setting: {request.setting}
        Tone: {request.tone}
        Art style: {request.art_style}
        Return exactly five panels. Each panel needs panel_number, title, scene_description and image_prompt.
        Keep character appearance and story continuity consistent. Do not place written text inside image prompts."""
        data = self._generate_json(prompt, settings.gemini_outline_model, ComicOutline)
        return ComicOutline.model_validate(data)

    def create_story(self, outline: ComicOutline, request) -> ComicStory:
        prompt = f"""Write the final script for this five-panel comic.
        Character: {request.character_name}
        Tone: {request.tone}
        Outline: {outline.model_dump_json()}
        Return exactly five panels. Each needs panel_number, caption, narration and short natural dialogue.
        Keep continuity and make panel five feel like a satisfying ending."""
        data = self._generate_json(prompt, settings.gemini_story_model, ComicStory)
        return ComicStory.model_validate(data)
```

app/services/comic_service.py

```
from app.config import settings
from app.schemas import ComicOutline, ComicStory, ComicPanel
from app.services.gemini_service import GeminiService
from app.services.image_generator import generate_image

class ComicService:
    def __init__(self):
        self.ai = GeminiService()

    def _demo_outline(self, request):
        beats = [
            ("The Spark", f"{request.character_name} discovers an unusual glowing clue in {request.setting}."),
            ("The Journey", f"{request.character_name} follows the clue through {request.setting}, growing curious and excited."),
            ("The Twist", f"The clue reveals a surprising obstacle that changes the plan."),
            ("The Choice", f"{request.character_name} makes a brave choice and solves the problem creatively."),
            ("A New Beginning", f"The adventure ends with a warm discovery and a promise of another story."),
        ]
        return ComicOutline(title="The Five-Panel Adventure", panels=[
            {"panel_number": i, "title": t, "scene_description": d, "image_prompt": f"{request.art_style}, {d}, consistent character design, cinematic composition"}
            for i, (t, d) in enumerate(beats, 1)
        ])

    def _demo_story(self, outline, request):
        lines = [
            ("A strange light appears.", "Something just lit up!", "Whoa... what is that?"),
            ("Curiosity takes the lead.", "Mira follows the glowing trail.", "Okay, mystery. I'm coming!"),
            ("Nothing goes exactly to plan.", "The trail suddenly disappears behind a locked gate.", "You have got to be kidding me."),
            ("A clever choice changes everything.", "Instead of forcing the gate, Mira notices a tiny side path.", "Sometimes the smallest door is the right one."),
            ("Every adventure starts somewhere.", "Beyond the path waits a bright new beginning.", "I have a feeling this is only chapter one."),
        ]
        return ComicStory(title=outline.title, panels=[
            {"panel_number": i, "caption": c, "narration": n, "dialogue": d} for i, (c, n, d) in enumerate(lines, 1)
        ])

    def generate(self, request):
        if settings.demo_mode or not settings.gemini_api_key:
            outline = self._demo_outline(request)
            story = self._demo_story(outline, request)
        else:
            outline = self.ai.create_outline(request)
            story = self.ai.create_story(outline, request)
        story_by_num = {p.panel_number: p for p in story.panels}
        panels = []
        for p in outline.panels[:settings.panel_count]:
            s = story_by_num[p.panel_number]
            image_url = generate_image(p.panel_number, p.title, p.image_prompt)
            panels.append(ComicPanel(
                panel_number=p.panel_number, title=p.title,
                scene_description=p.scene_description, image_prompt=p.image_prompt,
                caption=s.caption, narration=s.narration, dialogue=s.dialogue,
                image_url=image_url,
            ))
        return story.title, panels
```

app/services/image_generator.py

```
from pathlib import Path
from uuid import uuid4
from PIL import Image, ImageDraw, ImageFont
from app.config import settings

OUT = Path("app/static/panels")
OUT.mkdir(parents=True, exist_ok=True)

def _demo_image(number: int, title: str, prompt: str) -> str:
    path = OUT / f"panel_{number}_{uuid4().hex[:8]}.png"
    img = Image.new("RGB", (1200, 760), "#f4eee5")
    draw = ImageDraw.Draw(img)
    draw.rectangle((35, 35, 1165, 725), outline="#342b28", width=8)
    draw.ellipse((420, 155, 780, 515), fill="#e8c7a9", outline="#342b28", width=5)
    draw.ellipse((505, 265, 545, 305), fill="#342b28")
    draw.ellipse((655, 265, 695, 305), fill="#342b28")
    draw.arc((535, 310, 665, 405), 10, 170, fill="#342b28", width=6)
    draw.polygon([(470, 520), (730, 520), (820, 700), (380, 700)], fill="#d9b6c8", outline="#342b28")
    draw.text((70, 65), f"PANEL {number}: {title}", fill="#342b28")
    draw.text((70, 650), "ComicCraft AI • Demo artwork", fill="#342b28")
    img.save(path)
    return f"/static/panels/{path.name}"

def generate_image(number: int, title: str, prompt: str) -> str:
    if not settings.hf_token:
        if settings.demo_mode:
            return _demo_image(number, title, prompt)
        raise RuntimeError("HF_TOKEN is not configured.")
    try:
        from huggingface_hub import InferenceClient
        client = InferenceClient(api_key=settings.hf_token, provider="auto")
        image = client.text_to_image(prompt, model=settings.hf_image_model)
        path = OUT / f"panel_{number}_{uuid4().hex[:8]}.png"
        image.save(path)
        return f"/static/panels/{path.name}"
    except Exception:
        if settings.demo_mode:
            return _demo_image(number, title, prompt)
        raise
```

app/services/exporters.py

```
from pathlib import Path
from uuid import uuid4
from fpdf import FPDF

EXPORT_DIR = Path("app/static/exports")
EXPORT_DIR.mkdir(parents=True, exist_ok=True)

def export_pdf(title, panels):
    filename = f"comic_{uuid4().hex[:10]}.pdf"
    path = EXPORT_DIR / filename
    pdf = FPDF(orientation="P", unit="mm", format="A4")
    for panel in panels:
        pdf.add_page()
        pdf.set_font("Helvetica", "B", 18)
        pdf.multi_cell(0, 10, f"{title} – Panel {panel.panel_number}")
        image_path = Path(panel.image_url.lstrip("/"))
        if image_path.exists():
            pdf.image(str(image_path), x=10, y=35, w=190)
        pdf.set_y(145)
        pdf.set_font("Helvetica", "B", 13)
        pdf.multi_cell(0, 8, panel.title)
        pdf.set_font("Helvetica", size=11)
        pdf.multi_cell(0, 7, panel.caption)
        pdf.multi_cell(0, 7, panel.narration)
        pdf.multi_cell(0, 7, f"Dialogue: {panel.dialogue}")
    pdf.output(str(path))
    return f"/static/exports/{filename}"
```

app/templates/index.html

```
<!doctype html><html><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
scale=1"><title>ComicCraft AI</title><link rel="stylesheet" href="/static/css/style.css"></head><body><main
class="wrap"><section class="hero"><p class="badge">♦ AI POWERED</p><h1>ComicCraft AI</h1><p>Turn your imagination
into an original five-panel comic story.</p></section><form action="/generate" method="post"
class="card"><label>Story idea</label><textarea name="story_prompt" required placeholder="Example: A shy girl
discovers a magical cat that can talk..."></textarea><div class="grid"><label>Character name<input
name="character_name" value="Mira"></label><label>Setting<input name="setting" value="a colorful modern city"></
label><label>Tone<input name="tone" value="fun and adventurous"></label><label>Art style<input name="art_style"
value="vibrant cinematic comic art"></label></div><button type="submit">■ Create My Comic</button></form></main></
body></html>
```

app/templates/comic_preview.html

```
<!doctype html><html><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
scale=1"><title>{{ title }}</title><link rel="stylesheet" href="/static/css/style.css"></head><body><main
class="wrap"><div class="top"><div><p class="badge">YOUR COMIC</p><h1>{{ title }}</h1></div><form action="/export"
method="post"><button>■ Export PDF</button></form></div><section class="comic">{% for p in panels %}<article
class="panel"><div class="paneltext"><h2>{{ p.
panel_number }}. {{ p.title }}</h2><p><b>{{ p.caption }}</b></p><p>{{ p.narration }}</p><p class="dialogue">{{ p.
dialogue }}</p></div></article>{% endfor %}</section><a class="back" href="/"><← Create another comic</a></main></
body></html>
```

app/templates/export_success.html

```
<!doctype html><html><head><meta charset="utf-8"><title>Export Ready</title><link rel="stylesheet" href="/static/css/style.css"></head><body><main class="wrap center"><div class="card"><h1>■ PDF Ready!</h1><p>Your ComicCraft comic has been exported.</p><a class="button" href="{ url }">Download PDF</a><p><a href="/">Create another comic</a></p></div></main></body></html>
```

app/templates/error.html

```
<!doctype html><html><head><meta charset="utf-8"><title>Error</title><link rel="stylesheet" href="/static/css/style.css"></head><body><main class="wrap center"><div class="card"><h1>Something went wrong ■</h1><p>{{ error }}</p><a href="/">← Back</a></div></main></body></html>
```

app/static/css/style.css

```
*{box-sizing:border-box}body{margin:0;font-family:Inter,Arial,sans-serif;background:#f7f1e9;color:#292321}.wrap{width:min(1100px,92%);margin:0 auto;padding:45px 0}.hero{text-align:center;padding:30px}.hero h1,h1{font-size:clamp(2.2rem,6vw,4rem);margin:8px 0}.hero p{font-size:1.1rem;color:#665b55}.badge{display:inline-block;letter-spacing:2px;font-size:.75rem!important;font-weight:700;color:#8b596d}.card{background:#fff;border:1px solid #eadfd5;border-radius:22px;padding:28px;box-shadow:0 12px 35px #5a3c1a12}.card label,.grid label{display:block;font-weight:700;margin-bottom:15px}.grid{display:grid;grid-template-columns:1fr 1fr;gap:18px;margin-top:18px}textarea,input{width:100%;margin-top:8px;padding:14px;border:1px solid #d8cbc1;border-radius:12px;font:inherit;background:#fff}textarea{min-height:150px;resize:vertical}button,.button{display:inline-block;margin-top:20px;padding:13px 22px;border:0;border-radius:999px;background:#6f4658;color:white;font-weight:700;text-decoration:none;cursor:pointer}.top{display:flex;justify-content:space-between;align-items:center;gap:20px;margin-bottom:25px}.top h1{font-size:2.5rem}.comic{display:grid;gap:25px}.panel{background:#fff;border-radius:20px;overflow:hidden;border:1px solid #eadfd5;box-shadow:0 10px 25px #5a3c1a12}.panel img{display:block;width:100%;height:auto}.paneltext{padding:20px}.paneltext h2{margin-top:0}.dialogue{font-style:italic}.back{display:inline-block;margin-top:25px;color:#6f4658}.center{text-align:center}.center .card{max-width:600px;margin:auto}@media(max-width:700px){.grid{grid-template-columns:1fr}.top{display:block}}
```

app/static/js/app.js

```
document.addEventListener('DOMContentLoaded', () => {  
  const form = document.querySelector('form[action="/generate"]');  
  if (!form) return;  
  form.addEventListener('submit', () => {  
    const button = form.querySelector('button[type="submit"]');  
    if (button) { button.disabled = true; button.textContent = '■ Creating your comic...'; }  
  });  
});
```

tests/test_app.py

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    r = client.get('/health')
    assert r.status_code == 200
    assert r.json()['status'] == 'ok'

def test_home():
    r = client.get('/')
    assert r.status_code == 200
    assert 'ComicCraft AI' in r.text

def test_generate_json_demo():
    r = client.post('/generate-comic/json', json={'story_prompt': 'A girl finds a magical cat.'})
    assert r.status_code == 200
    data = r.json()
    assert len(data['panels']) == 5
```

pytest.ini

```
[pytest]
pythonpath = .
```

README.md

ComicCraft AI

ComicCraft AI turns a story idea into a five-panel comic using Gemini for story generation and Hugging Face text-to-image generation. It also supports demo mode and PDF export.

Quick start

```
```bash
python -m venv .env
Windows: .env\Scripts\activate
macOS/Linux: source .env/bin/activate
pip install -r requirements.txt
copy .env.example .env # Windows
cp .env.example .env # macOS/Linux
python run.py
```
```

Open <http://127.0.0.1:8000>

Demo mode

The default `.env.example` uses `DEMO_MODE=true`, so the app can be demonstrated without API keys. It creates deterministic demo artwork and a five-panel story.

Real AI mode

Set `DEMO_MODE=false`, then add your Gemini API key and Hugging Face token to `.env`. Keep secrets only in `.env`; never upload that file to GitHub.

Useful endpoints

- `/`` – web application
- `/generate`` – HTML comic generation
- `/generate-comic/json`` – JSON API
- `/export`` – PDF export
- `/health`` – health check
- `/docs`` – FastAPI API documentation

Test

```
```bash
pytest -q
```
```

SETUP.md

ComicCraft AI — VS Code Setup

1. Extract the ZIP and open the `ComicCraft-AI-Source` folder in VS Code.
2. Open the VS Code terminal.
3. Create a virtual environment:
`python -m venv .venv`
4. Activate it.
5. Install dependencies:
`pip install -r requirements.txt`
6. Copy `.env.example` to `.env`.
7. For a classroom demo, leave `DEMO\_MODE=true`.
8. Run:
`python run.py`
9. Open `http://127.0.0.1:8000`.
10. Enter a story idea and click **Create My Comic**.
11. Click **Export PDF** after generation.

For real Gemini + image generation, set `DEMO\_MODE=false` and add the API credentials in `.env`.

start_windows.bat

```
@echo off  
python -m uvicorn app.main:app --reload
```

start_linux.sh

```
#!/bin/sh  
python -m uvicorn app.main:app --reload
```

.gitignore

```
.env
__pycache__/
.pytest_cache/
*.pyc
.venv/
venv/
app/static/panels/*
!app/static/panels/.gitkeep
app/static/exports/*
!app/static/exports/.gitkeep
```